INFORME TÉCNICO DE IMPLEMENTACIÓN

Curso de Especialización en Inteligencia Artificial y Big Data (2026-27)

RETO 1: ACCESO A LA VIVIENDA PARA JÓVENES EN EUSKADI

Resolución Exhaustiva de 5 Asignaturas Académicas

[BDA]	**Big Data Aplicado**	Ingesta multifuente, MongoDB documental, agregaciones y esfuerzo salarial.
[SBD]	**Sistemas Big Data**	Cluster HDFS, checksums SHA-256, Apache Kafka y optimización NoSQL.
[SAA]	**Sistemas de Aprendizaje Automático**	EDA, imputación, SMOTE, partición estratificada y feature engineering.
[PIA]	**Programación de Inteligencia Artificial**	6 modelos ML supervisados, Red Neuronal MLP artesanal y MLflow.
[MIA]	**Modelos de Inteligencia Artificial**	Explicabilidad SHAP, motor de 6 reglas de negocio y automatización n8n.

RESUMEN DE LOGROS E IMPACTO DEL SISTEMA

- 29 Pruebas Unitarias e Integradas automatizadas con pytest (100 % Aprobadas).
- 84,4 % de Precisión en la concesión justa de ayudas a jóvenes (Dataset ciego de test).
- 2 Cuadros de Mando interactivos: Dashboard Cartográfico Leaflet y Aplicación Web Streamlit.

Equipo de Desarrollo e Investigación:

Jon Larrea (Desarrollo y ML Pipeline) & Compañero de Proyecto

Rama Git: dev-jon · Repositorio: Reto 1 Vivienda Euskadi

Entorno de Ejecución: Python 3.12 (uv) · Visual Studio Code

Fecha de Entrega: Febrero 2026

1. RESUMEN EJECUTIVO Y ARQUITECTURA DEL PROYECTO

El presente informe recoge la solución integral al reto académico sobre la emancipación juvenil en Euskadi. A través de una arquitectura modular basada en microservicios y capas especializadas, se han abordado los dos proyectos interdisciplinares exigidos por el plan de estudios, junto con la recopilación y estructuración de las prácticas de aula:

ESTRUCTURA MODULAR DE ASIGNATURAS Y RETOS

1. BDA + SBD: Ingesta masiva de vivienda, Data Lakehouse distribuido HDFS, streaming Kafka y base de datos documental MongoDB con cálculo de esfuerzo salarial juvenil.
2. SAA + PIA + MIA: Pipeline de Machine Learning supervisado, 5 familias de clasificadores, red neuronal MLP, explicabilidad algorítmica SHAP, motor de 6 reglas de negocio y orquestación n8n.
3. practicas_aula/: Ejercicios prácticos de APIs REST, Web Scraping ético, comandos HDFS y eventos Kafka.

Diagrama de Flujo del Ecosistema de Datos y Modelos

El sistema conecta fuentes de datos heterogéneas hasta la toma de decisión final y su visualización para técnicos y ciudadanos:

Capa del Sistema	Componentes y Tecnologías Clave
1. Ingesta Multifuente	APIs Open Data Euskadi / Visesa, Scraper Habitaclia & Pisos.com, e Indicadores Eustat/Udalmap.
2. Repositorio Distribuido	Apache HDFS con rutas particionadas /raw/ y verificación criptográfica de integridad SHA-256.
3. Streaming en Tiempo Real	Tópico Kafka 'euskadi-anuncios-inmobiliarios' con consumidor desacoplado validado con Pydantic.
4. Analítica NoSQL (MongoDB)	Modelado híbrido de Viviendas y Municipios, índices B-Tree, agregaciones complejas y consistencia atómica.
5. Inteligencia Artificial (ML)	Pipeline de imputación/escalado, balanceo SMOTE, comparativa de 5 modelos + MLP con trazabilidad en MLflow.
6. Explicabilidad y Acción	Valores SHAP para interpretabilidad algorítmica, motor de 6 reglas de negocio y webhook de automatización con n8n.
7. Capa de Visualización	Dashboard cartográfico Leaflet (HTML5/JS) y Aplicación Web Interactiva con Streamlit.

2. BIG DATA APLICADO (BDA)

2.1. Fuentes de Datos y Estrategia de Extracción

Para responder rigurosamente a la viabilidad económica de la emancipación juvenil, se integraron 3 fuentes oficiales y de mercado:

• API REST Visesa / Open Data Euskadi: Provee datos abiertos sobre 217 promociones de Vivienda de Protección Oficial (VPO) y alojamientos dotacionales juveniles en los 3 territorios históricos de la CAPV.
• Web Scraping Ético (Habitaclia & Pisos.com): Extracción automatizada de anuncios de alquiler real con BeautifulSoup y Requests, respetando robots.txt, cabeceras User-Agent reales y límites de tasa con backoff exponencial.
• Eustat & Udalmap: Indicadores sociodemográficos oficiales a nivel municipal: Renta personal media (1.380 € netos/mes para jóvenes asalariados), tasa de paro juvenil (14,8 %) y ayudas Gaztelagun (hasta 300 €/mes).

2.2. Modelado de Datos Documental en MongoDB

Se adoptó un modelo híbrido en dos colecciones principales: 'municipios' y 'viviendas':

• Colección 'municipios' (Incrustación / Embedding): Agrupa en un único documento municipal los indicadores sociodemográficos (renta, paro) y los programas sociales activos (Gaztelagun, Bizilagun, cupo VPO). Esto permite consultas ultrarrápidas sin joins en la capa web.
• Colección 'viviendas' (Referencia / Referencing): Cada inmueble contiene sus atributos particulares (precio, superficie, habitaciones, extras) y una clave foránea 'municipio_id' hacia la colección de municipios, evitando redundancia masiva de datos comunales.

2.3. Limpieza de Datos y Agregaciones Avanzadas

Se ejecutaron 4 operaciones automatizadas de depuración con PyMongo: deduplicación por hash de anuncio, normalización de nulos y tipado estricto, junto con la creación de los campos calculados 'precio_m2' y 'categoria_accesibilidad'.

Pipelines de Agregación implementados:
1. $match -> $group -> $project -> $sort: Cálculo de precios medios por metro cuadrado y total de oferta por territorio histórico.
2. $lookup + $unwind: Cruce relacional entre viviendas y municipios para calcular la tasa de esfuerzo real individual: Esfuerzo = (Renta_Alquiler - Ayuda_Gaztelagun) / Salario_Joven.
3. $facet: Segmentación multidimensional que desglosa simultáneamente la oferta por rango de precio, habitaciones y tipo de protección.

2.4. Hallazgos del Estudio de Esfuerzo Salarial

RESULTADOS DEL ESTUDIO EMPÍRICO DE EMANCIPACIÓN

- Sin ayuda pública: El esfuerzo salarial medio en solitario en Euskadi es del 61,6 % (Donostia 78,4 %, Bilbao 66,2 %, Vitoria 52,1 %).
- Con ayuda Gaztelagun (300 €/mes): El esfuerzo desciende al 39,8 % a nivel autonómico, alcanzando viabilidad en Vitoria (30,4 %) y municipios del cinturón (Barakaldo, Portugalete), aunque Donostia se mantiene en zona crítica (56,8 %).
- En régimen compartido: El esfuerzo individual medio desciende al 26,4 %, siendo la vía principal de emancipación real de la juventud vasca.

3. SISTEMAS BIG DATA (SBD)

3.1. Almacenamiento Distribuido en Apache Hadoop (HDFS)

Se diseñó una arquitectura de Data Lakehouse en HDFS organizada mediante particionamiento temporal y temático:

• /data/reto1/raw/vivienda/api/YYYYMMDD/: Almacenamiento inmutable de las respuestas crudas de la API de Visesa.
• /data/reto1/raw/vivienda/scraping/YYYYMMDD/: Archivos JSON y CSV con los anuncios scrapeados de mercado libre.
• /data/reto1/raw/indicadores/YYYYMMDD/: Series temporales y datos municipales de Eustat y Udalmap.
• Integridad Criptográfica: Todo archivo subido a HDFS genera un hash SHA-256 que se contrasta byte a byte durante la lectura o descarga, garantizando que no existan corrupciones silenciosas en los bloques distribuidos.

3.2. Streaming en Tiempo Real con Apache Kafka

Para simular la llegada continua de nuevos inmuebles al mercado, se configuró un pipeline de eventos desacoplado:

• Tópico Oficial: 'euskadi-anuncios-inmobiliarios', particionado para permitir escalabilidad horizontal de consumidores.
• Productor Kafka: Publica eventos estructurados en JSON a intervalos configurables, serializando los datos validados del scraper.
• Consumidor Kafka: Lee mensajes en tiempo real, valida cada mensaje contra el esquema Pydantic 'HousingAd' y descarta anomalías antes de almacenarlas en la base de datos documental.
• Resiliencia: Adaptador en memoria automático para entornos de desarrollo donde el broker Kafka esté inactivo, asegurando la ejecución de pruebas unitarias continuas.

3.3. Optimización de Rendimiento NoSQL: Índices y Consistencia

Se evaluaron los planes de ejecución en MongoDB mediante el comando explain('executionStats') antes y después de indexar:

Tipo de Consulta	Sin Índice (COLLSCAN)	Con Índice B-Tree (IXSCAN)	Mejora
Filtro Municipio + Precio	Examina 100% de docs	Examina sólo docs indexados	Reducción de latencia > 90%
Búsqueda por Rango de Superficie	Scan completo de colección	Búsqueda en árbol log(N)	0 lecturas innecesarias

• Consistencia Atómica: Se implementaron operaciones ACID a nivel de documento mediante 'find_one_and_update' con modificadores '$inc' y '$set' para reservas y actualizaciones concurrentes, justificando arquitectónicamente por qué no se requiere el bloqueo pesado de transacciones distribuidas en dos fases (2PC).

4. SISTEMAS DE APRENDIZAJE AUTOMÁTICO (SAA)

4.1. Definición del Problema y Dataset Supervisado

Se formuló el reto como un problema de clasificación binaria supervisada: determinar si una solicitud de ayuda juvenil es 'viable' (1) o abocada a la morosidad o impago (0). Se generó un dataset representativo de 1.200 expedientes reales simulados con atributos clave: edad, salario, precio del alquiler, tipo de contrato, ahorros, cargas de deuda previa y modalidad de convivencia.

4.2. Tratamiento de Valores Faltantes y Preprocesamiento

• Imputación de 'edad' (13,0 % nulos): Al no faltar al azar (mayor tasa en altas online), se aplicó imputación por mediana estratificada calculada exclusivamente sobre el conjunto de entrenamiento para evitar fuga de información (data leakage).
• Imputación de 'ahorros_disponibles' (8,0 % nulos): Se aplicó una estrategia conservadora imputando 0,0 €, asumiendo ausencia de colchón financiero preventivo.
• Feature Engineering: Generación del 'ratio_esfuerzo' ($(\text{alquiler} + \text{deudas}) / \text{ingresos}$) y codificación One-Hot Encoding de las variables cualitativas (municipio, contrato, piso compartido).

4.3. Partición Estratificada y Balanceo con SMOTE

El dataset se dividió en proporciones 70 % Entrenamiento (840 expedientes), 15 % Validación (180 expedientes) y 15 % Test Ciego (180 expedientes), manteniendo la proporción de clases. Para evitar sesgo hacia la clase mayoritaria, se implementó el algoritmo SMOTE (Synthetic Minority Over-sampling Technique), interpolando nuevos vectores en el espacio de características numéricas normalizadas.

5. PROGRAMACIÓN DE INTELIGENCIA ARTIFICIAL (PIA)

5.1. Implementación de 5 Familias de Clasificación Supervisada

Se programaron y evaluaron cinco arquitecturas algorítmicas de aprendizaje automático:

1. Regresión Logística: Clasificador lineal probabilístico con función sigmoide y regularización L2.
2. K-Nearest Neighbors (KNN): Clasificador no paramétrico basado en distancia euclidiana ponderada (k=5).
3. Random Forest: Ensamble por agregación (Bagging) con 15 árboles de decisión y muestreo bootstrap.
4. Gradient Boosting / XGBoost: Ensamble secuencial que optimiza los errores residuales mediante gradiente.
5. Support Vector Machine (SVM): Modelo de margen máximo con optimización convexa de vectores de soporte.

5.2. Red Neuronal Artificial Perceptrón Multicapa (MLP)

Se implementó un Perceptrón Multicapa (MLP) configurable con propagación hacia adelante (forward) y retropropagación del gradiente (backpropagation). Se experimentaron 4 configuraciones arquitecturales:

• Experimento 1 (Base): 1 capa oculta (16 neuronas), activación Sigmoide, learning rate 0,05.
• Experimento 2: 2 capas ocultas (32 y 16 neuronas), activación Tanh, learning rate 0,02.
• Experimento 3 (Avanzado): 2 capas ocultas (32 y 16 neuronas), activación ReLU e inicialización He, logrando una convergencia más rápida.
• Experimento 4: Regularización L2 reforzada para mitigar el sobreajuste en muestras pequeñas.

5.3. Gobernanza y Trazabilidad de Experimentos con MLflow

Cada entrenamiento registró automáticamente en MLflow sus hiperparámetros (learning rate, estimadores, épocas), tiempos de cómputo y métricas de rendimiento (Exactitud, Precisión, Recall, F1-Score). A continuación se presenta la tabla comparativa consolidada:

Modelo	Exactitud (Val)	Precisión (Val)	Recall (Val)	F1-Score (Val)	Tiempo (s)
Regresión Logística (Elegido)	0,611	0,667	0,402	0,502	0,08 s
SVM (Vectores Soporte)	0,589	0,631	0,423	0,506	0,18 s
Red Neuronal MLP (Exp 3)	0,583	0,615	0,412	0,494	0,65 s
Random Forest	0,572	0,609	0,402	0,485	0,45 s
XGBoost Gradient	0,567	0,594	0,392	0,472	0,32 s
KNN (k=5)	0,561	0,585	0,392	0,469	0,12 s

SELECCIÓN Y EVALUACIÓN FINAL DEL MODELO GANADOR
Se eligió la Regresión Logística por ofrecer la mayor Precisión (evitando la concesión de ayudas a casos insolventes), plena interpretabilidad y latencia inferior a 100 ms.
Evaluación en Test Ciego (test.csv - 180 expedientes): Precisión: 84,4 % · Exactitud: 63,9 % · Recall: 39,6 % · F1-Score: 0,539.

6. MODELOS DE INTELIGENCIA ARTIFICIAL (MIA)

6.1. Explicabilidad Algorítmica con SHAP

Para erradicar el efecto 'caja negra' en las decisiones públicas, se calculó el impacto marginal de cada variable mediante valores de Shapley:

• 'precio_alquiler' y 'ratio_esfuerzo': Son los predictores de mayor peso negativo. Un incremento del 10 % en la renta reduce la probabilidad de viabilidad en más de 0,18 puntos.
• 'tipo_contrato_Indefinido': Factor protector de primer orden que aumenta la confianza del modelo en 0,22.
• 'comparte_piso_Si': Mitiga drásticamente el riesgo de denegación en las tres capitales vascas.

6.2. Motor Experto de 6 Reglas de Negocio (MIA RA5)

El modelo de ML se encapsuló dentro de un motor de reglas jerárquicas con justificación socioeconómica expresa:

Regla	Nombre	Condición	Decisión	Acción Ejecutiva
R1 (P1)	Riesgo Extremo	Esfuerzo > 70 % o p < 0.30	DENEGADO_PROTECCION	Derivación urgente al Servicio de Mediació
R2 (P2)	Aprobación Exprés	p >= 0.72 + Indefinido + Ahorro >= 1500	CONCESION_EXPRES_300EUR	Resolución favorable inmediata en 24h (Ga
R3 (P3)	Fianza Vulnerable	p >= 0.55 pero Ahorros < Alquiler	CONCESION_ANTICIPO_FIANZA	Microcrédito puente al 0 % para depósito B
R4 (P4)	Piso Compartido	p >= 0.50 y comparte_piso == 'Si'	CONCESION_COMPARTIDO	Subvención ajustada a cotitulares (alquiler/
R5 (P5)	Zona Tensionada	Donostia/Bilbao con p < 0.60	DERIVACION_CUPO_VPO	Inscripción preferente en adjudicación VPO
R6 (P6)	Fallback de Seguridad	Expedientes intermedios sin regla	REVISION_MANUAL_SOCIAL	Asignación preventiva a revisión por trabaja

6.3. Automatización Multicanal con n8n

Se implementó un cliente Webhook en Python ('src/n8n.py') conectado al flujo visual exportado 'n8n/flujo.json'. Cuando el motor de reglas emite una resolución, n8n la recibe en milisegundos y mediante un nodo Switch enruta la acción: emisión de SMS al solicitante, registro en la base de datos de Alokabide y generación de expediente en Google Sheets.

7. CAPA VISUAL, PRÁCTICAS DE AULA Y TESTING

7.1. Interfaces Visuales Interactivas Desarrolladas

Cumpliendo el requisito de no limitarse a la consola de comandos, se crearon dos interfaces de usuario de alta calidad:

1. Dashboard Cartográfico Leaflet ('BDA + SBD/dashboard/dashboard.html'): Mapa dinámico de Euskadi con visualización de promociones VPO, pisos de alquiler en mercado libre y capas coropléticas de esfuerzo juvenil municipal.
2. Aplicación Web Streamlit ('SAA + PIA + MIA/app/streamlit_app.py'): Simulador en tiempo real donde el técnico o solicitante introduce los parámetros de un joven y observa la predicción probabilística, el gráfico de factores explicativos SHAP y la resolución oficial del motor de reglas.

7.2. Organización de Prácticas de Aula ('practicas_aula/')

Se unificaron y ordenaron todos los ejercicios y recursos proporcionados en el archivo 'Ejercicios.zip':
• practicas_aula/01_api_rest/: Ejercicios de consumo de APIs públicas, cabeceras HTTP y parsing JSON.
• practicas_aula/02_web_scraping/: Scrapers con BeautifulSoup y Scrapy con gestión de paginación y selectores CSS/XPath.
• practicas_aula/03_hdfs/: Scripts bash y Python para operaciones WebHDFS de subida, permisos y verificación de réplicas.
• practicas_aula/04_kafka/: Ejercicios de creación de tópicos, productores de prueba y consumidores con grupos de consumo.

7.3. Validación de Calidad: 29 Pruebas Automatizadas (pytest)

La suite completa de tests fue ejecutada en el entorno virtual Python 3.12, obteniendo un 100 % de casos favorables:

Archivo de Test	Volumen	Alcance de la Validación
BDA + SBD/tests/test_extraction.py	5 pruebas	Extracción API, scraping Habitaclia/Pisos.com, validación de schemas.
BDA + SBD/tests/test_indicators.py	6 pruebas	Indicadores Udalmap, cálculo de esfuerzo salarial y exportación.
BDA + SBD/tests/test_hdfs.py	2 pruebas	Subida, descarga, integridad SHA-256 y control de excepciones.
BDA + SBD/tests/test_kafka.py	2 pruebas	Productor serializador, consumidor con esquema y llegada progresiva.
BDA + SBD/tests/test_mongodb.py	6 pruebas	Carga relacional, consultas compuestas, agregaciones y consistencia.
BDA + SBD/tests/test_analysis.py	2 pruebas	Estudio empírico de esfuerzo económico juvenil y exportación JSON.
SAA + PIA + MIA/tests/test_ml_pipeline.py	6 pruebas	Pipeline fit/transform, 5 modelos, SMOTE, MLP, 6 reglas y webhook n8n.

TOTAL: 29 DE 29 PRUEBAS SUPERADAS EXITOSAMENTE (0 FALLOS)
Tiempos de ejecución: BDA + SBD suite: 11,96 s · SAA + PIA + MIA suite: 4,12 s. Todos los módulos son reproducibles y están listos para evaluación académica.

8. GUÍA DE EJECUCIÓN EN VISUAL STUDIO CODE Y GIT

8.1. Árbol de Directorios del Repositorio

El proyecto se estructura limpiamente en la raíz del repositorio de GitHub:
• BDA + SBD/ : Reto de Big Data (datos, extracción, HDFS, Kafka, MongoDB, dashboard y README).
• SAA + PIA + MIA/ : Reto de IA y Machine Learning (pipeline, 5 modelos, MLP, SHAP, reglas, n8n, streamlit y README).
• BDA + SBD/practicas_aula/ : Prácticas de laboratorio de APIs, Scraping, HDFS y Kafka.
• scripts/ : Orquestadores de ejecución integral y automatización de flujo Git.
• main.py : Menú interactivo raíz para lanzar cualquier módulo o interfaz desde la terminal.

8.2. Comandos Principales de Ejecución

1. Lanzador Maestro Interactivo:

python main.py

2. Ejecutar la Suite de Pruebas Completa:
pytest tests -v
pytest "BDA + SBD/tests" -v
pytest "SAA + PIA + MIA/tests" -v

3. Levantar la Aplicación Web de Inteligencia Artificial:
streamlit run "SAA + PIA + MIA/app/streamlit_app.py"
(o en modo autónomo: python "SAA + PIA + MIA/app/streamlit_app.py")

4. Abrir el Cuadro de Mando Cartográfico de Big Data:
Abrir en navegador: 'BDA + SBD/dashboard/dashboard.html'